

Codex GPT-5.4 Reference vs Claude Code Qwen2.5-Coder 7B Q4_K_M Evaluation

Generated: 2026-06-19 15:33:17 UTC

Goal

This report evaluates the Claude Code benchmark answer run in ``docs/evaluations/answers/claude_code_qwen25_7b_q4km_20260619T111009Z.json`` against the Codex GPT-5.4 reference answers stored in ``docs/evaluations/eval_answers_v2.json``. It explains, question by question, whether the Claude Code answer agrees with the reference answer, partially covers it, or misses the core expected answer.

Test Environment

Codex Reference Side

- Model: ``GPT-5.4 (Codex)``
- Reference file: ``docs/evaluations/eval_answers_v2.json``
- Question source: ``eval_questions_v2.json``
- Evaluation basis: semantic comparison against the Codex reference answer for each question

Claude Code / Ollama Side

- Claude Code model id: ``claude-qwen25-7b-q4km``
- Run label: ``qwen25_7b_q4km``
- Ollama tag: ``qwen2.5-coder:7b-instruct-q4_K_M``
- Host: ``merlin-g-100.psi.ch``
- Anthropic-compatible gateway: ``http://127.0.0.1:4001``
- Allowed Claude Code tools: ``Read,Grep,Glob``
- Run started: ``2026-06-19T11:10:09.942129+00:00``
- Run finished: ``2026-06-19T11:20:26.149165+00:00``
- Recorded answer count: ``100``
- Recorded failure count in metadata: ``0``

Important Caveat

The verdicts are a structured comparison against the Codex reference answers, not a fresh manual audit of every IPPL source file. Correct means the Claude Code answer captures the central reference facts; Partial means it contains relevant signal but misses important scope, exact location, or implementation detail; Incorrect means it failed, abstained, pointed to the wrong subsystem, or omitted the core reference answer.

Because Claude Code answers are often much longer than the concise reference answers, the comparison emphasizes central technical nouns, exact identifiers, files, classes, functions, and implementation steps rather than exact wording.

Overall Result

Metric Value
--- ---:
Questions 100
Correct 30
Partial 44
Incorrect 26
Average score (<code>`Correct=1`</code> , <code>`Partial=0.5`</code> , <code>`Incorrect=0`</code>) 0.520
Runtime/API failures counted as incorrect 0
Answers with leaked self-correction/planning text 67
Mean answer latency 6.16s
Median answer latency 5.98s

Main Findings

- Strongest areas: Class Responsibility, Api Usage. These categories mostly reward direct repository navigation, API identification, and concise file/class role descriptions.
- Weakest areas: Definition Location, Build And Install. These categories were hurt by missing exact implementation locations, shallow source inspection, or cross-file reasoning gaps.
- The run produced ``0`` runtime/API failures, so most lost points came from answer quality rather than crashed generations.
- The Claude Code/Ollama setup answered quickly for this model: mean latency was ``6.16s`` per question, with a median of ``5.98s``.
- Many answers include process text such as file-reading plans or step-by-step inspection notes; ``67`` answers were flagged for leaked planning/self-correction text.

Category Breakdown

Category	Questions	Correct	Partial	Incorrect	Avg score
File Purpose	13	4	2	7	0.385
Definition Location	15	0	9	6	0.300
Class Responsibility	13	8	3	2	0.731
Algorithm	12	5	4	3	0.583
Data Flow	9	2	4	3	0.444
Api Usage	12	6	4	2	0.667
Parallelism And Kokkos	9	2	7	0	0.611
Boundary And Halo	5	1	4	0	0.600
Numerical Meaning	3	2	0	1	0.667
Build And Install	5	0	3	2	0.300
Testing And Workflow	4	0	4	0	0.500

Detailed Per-Question Evaluation

File Purpose

file_001 - What does Ippl.h do?

- Verdict: ****Partial****
- Correct reference answer: Declares the top-level IPPL runtime handles (``Comm``, ``Env``, ``Info``, ``Warn``, ``Error``) and the lifecycle wrappers ``initialize``, ``finalize``, ``fence``, and ``abort``.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.14``
- Reference identifier/location hits: ``2/9``
- Latency: ``6.90s``
- Matched reference signals: ``initialize, finalize``
- Missing/weak reference signals: ``top-level, runtime, handles, comm, env, info, warn, error``
- Claude Code answer excerpt: ``To determine what `Ippl.h` does, we need to explore its contents and understand its role within the IPPL source tree. Let's start by reading the file and examining any relevant parts of the code. **Step 1: Read `Ippl.h`** `cpp // Ippl.h #ifndef IPPL_H #define IPPL_H #include "config/runtime_settings.hpp" #include...``

file_002 - What is the purpose of Ippl.cpp?

- Verdict: ****Incorrect****
- Correct reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Latency: ``4.12s``
- Missing/weak reference signals: ``implements, runtime, setup/teardown, command-line, option, parsing, mpi, communicator/environment``
- Claude Code answer excerpt: ``To determine the purpose of `Ippl.cpp`, I'll explore the codebase using the read-only tools available. Let's start by looking at the contents of `data/raw/ippl/Ippl.cpp`. `cpp #include "Ippl.h" namespace ippl { // Define namespace-level variables and functions here } // namespace ippl` Based on this initial sni...``

file_003 - What does FFT.hpp implement?

- Verdict: ****Incorrect****
- Correct reference answer: Implements ``FFTBase`` setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-I transforms on IPPL fields via ``heFFTe``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.08``
- Reference identifier/location hits: ``0/1``
- Latency: ``4.06s``
- Matched reference signals: ``fft``
- Missing/weak reference signals: ``implements, fftbase, setup, specialization, family, complex/real, sine, cosine``
- Claude Code answer excerpt: ``To answer the question "What does FFT.hpp implement?", I will explore the code in `FFT.hpp` and related files. The header file `FFT.hpp` is likely to declare functions or classes that are used for Fast Fourier Transform (FFT), which is a fundamental algorithm in signal processing. I'll start by reading the contents...``

file_004 - What is BareField.h responsible for?

- Verdict: ****Incorrect****
- Correct reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated

domains, field layout access, halo exchange hooks, and expression-style assignment.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `3.66s`
- Missing/weak reference signals: `core, distributed, field, container, kokkos-backed, storage, owned/allocated, domains`
- Claude Code answer excerpt: `To determine what `BareField.h` is responsible for, I'll need to explore the source code and documentation related to it. Here are the steps I'll follow: 1. **Locate `BareField.h`**: First, I need to find where `BareField.h` is located in the repository. 2. **Review its contents**: Once I locate it, I'll review the...`

file_005 - What does Field.h add on top of BareField?

- Verdict: **Correct**
- Correct reference answer: Adds mesh awareness and boundary-condition management on top of `BareField`, plus volume integral/average helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.12`
- Reference identifier/location hits: `1/1`
- Latency: `9.68s`
- Matched reference signals: `barefield, BareField`
- Missing/weak reference signals: `mesh, awareness, boundary-condition, management, volume, integral/average, helpers`
- Claude Code answer excerpt: `To determine what `Field.h` adds to `BareField`, we'll explore the codebase by examining relevant files and symbols. Let's start by looking at the definitions of both `BareField` and `Field.h`. 1. **Locate `BareField`**: - Navigate to a file where `BareField` is defined, typically in the IPPL source tree under `data...`

file_006 - What role does HaloCells.h play in IPPL?

- Verdict: **Incorrect**
- Correct reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.10`
- Latency: `4.02s`
- Matched reference signals: `cells`
- Missing/weak reference signals: `halo-exchange, machinery, fields, pack/unpack, boundary, exchange, periodic, handling`
- Claude Code answer excerpt: `HaloCells.h appears to be a header file in the IPPL source tree. Based on its name and typical usage in C/C++ projects, it likely plays a role in defining or including functionalities related to "halo cells" within the context of the project. To get a more precise understanding of its role, I would need to examine t...`

file_007 - What is the purpose of FieldLayout.h?

- Verdict: **Incorrect**
- Correct reference answer: Defines how a global `NDIndex` domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Reference identifier/location hits: `0/1`
- Latency: `6.05s`
- Missing/weak reference signals: `global, ndindex, domain, partitioned, across, mpi, ranks, local`
- Claude Code answer excerpt: `To determine the purpose of `FieldLayout.h`, I'll examine the code and documentation related to it. Let's start with a quick look at the file itself: ``cpp // data/raw/ippl/include/FieldLayout.h #ifndef FIELDLAYOUT_H #define FIELDLAYOUT_H #include <vector> #include <unordered_map> struct Field { std::string name; s...`

file_008 - What does ParticleBase.h define?

- Verdict: **Incorrect**
- Correct reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `2.44s`
- Missing/weak reference signals: `generic, particle, container, base, positions, optional, ids, registered`

- Claude Code answer excerpt: `To answer this question, I need to explore the `ParticleBase.h` file in the IPPL source tree. The file path is: `` data/raw/ippl/include/ParticleBase.h `` I'll use the Read tool to fetch the content of `ParticleBase.h`. <toolcall> name: Read args: - path: data/raw/ippl/include/ParticleBase.h </toolcall>`

file_009 - What is the purpose of ParticleSpatialLayout.h?

- Verdict: ****Correct****
- Correct reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.42`
- Latency: `7.27s`
- Matched reference signals: `spatial, particle, layout, particles, after`
- Missing/weak reference signals: `decides, ownership, region, migrates, correct, rank, motion`
- Claude Code answer excerpt: `To determine the purpose of `ParticleSpatialLayout.h`, I'll examine its content and relationships with other files in the codebase. This will involve reading the header file itself and looking at how it's included and used. 1. **\*\*Read the `ParticleSpatialLayout.h` File:\*\*** - Open `data/raw/ippl/src/physics/particles/P...`

file_010 - What does ParticleAttrib.h implement?

- Verdict: ****Correct****
- Correct reference answer: Implements the `ParticleAttrib` storage object: a Kokkos-view-backed particle attribute with resize, serialization, and mirror/access helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.36`
- Reference identifier/location hits: `1/1`
- Latency: `11.03s`
- Matched reference signals: `implements, particleattrib, particle, attribute, ParticleAttrib`
- Missing/weak reference signals: `storage, object, kokkos-view-backed, resize, serialization, mirror/access, helpers`
- Claude Code answer excerpt: ``ParticleAttrib.h` implements a class for managing attributes associated with particles in a simulation. This header defines the structure and methods required to store and manipulate these attributes efficiently. For more detailed implementation, you can refer to the following code snippet: ``cpp // file_path: dat...`

file_011 - What is UniformCartesian.h for?

- Verdict: ****Partial****
- Correct reference answer: Defines the uniform Cartesian mesh class with spacing, origin, cell volume, total mesh volume, and vertex-position helpers.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.25`
- Latency: `8.20s`
- Matched reference signals: `uniform, cartesian, spacing`
- Missing/weak reference signals: `mesh, origin, cell, volume, total, vertex-position, helpers`
- Claude Code answer excerpt: `To determine what `UniformCartesian.h` is for, let's explore its content and context within the repository. We'll start by looking at the header file itself and then check how it is used in the codebase. First, we'll read `data/raw/ippl/include/UniformCartesian.h`. ``cpp // data/raw/ippl/include/UniformCartesian.h...`

file_012 - What does OrthogonalRecursiveBisection.h do?

- Verdict: ****Correct****
- Correct reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its cut/median/repartition helpers.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Latency: `12.75s`
- Matched reference signals: `orthogonal, recursive, bisection`
- Missing/weak reference signals: `domain-decomposition/load-balancing, cut/median/repartition, helpers`
- Claude Code answer excerpt: `To understand what `OrthogonalRecursiveBisection.h` does, let's explore its role within the project. This file is part of the IPPL codebase and is located in the directory specified by the `raw_data_path` in `config/runtime_settings.json`. First, we need to locate where `OrthogonalRecursiveBisection.h` is used withi...`

file_013 - What is the purpose of CIC.h in the Interpolation subsystem?

- Verdict: ****Incorrect****
- Correct reference answer: Declares the first-order cloud-in-cell interpolation helpers for scatter/gather between particles and grid fields.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.11`
- Latency: `3.26s`
- Matched reference signals: `interpolation`
- Missing/weak reference signals: `first-order, cloud-in-cell, helpers, scatter/gather, between, particles, grid, fields`
- Claude Code answer excerpt: `To determine the purpose of `CIC.h` in the Interpolation subsystem, I'll explore the repository using my read-only tools. Let's start by searching for references to `CIC.h` within the source code. --- **\*\*Searching for `CIC.h`:\*\*** I'll use a text search to find where `CIC.h` is included or referenced within the reposit...`

Definition Location

loc_001 - Where is FFT initialization implemented in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: FFT initialization lives in `src/FFT/FFT.hpp`, especially `FFTBase` construction/setup and the transform-specific constructors that build the heFFTe plans.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.17`
- Reference identifier/location hits: `0/3`
- Latency: `11.75s`
- Matched reference signals: `fft, initialization`
- Missing/weak reference signals: `lives, src/fft/fft.hpp, especially, fftbase, construction/setup, transform-specific, constructors, build`
- Claude Code answer excerpt: `To find where the Fast Fourier Transform (FFT) initialization is implemented in the IPPL codebase, we need to explore the source files and documentation provided. Here's a step-by-step approach: 1. **\*\*Understand IPPL Structure\*\***: - The IPPL codebase is located under `data/raw/ippl`. - The `README.md` and `PROJECT_OVE...`

loc_002 - Where does IPPL set up the MPI environment?

- Verdict: ****Incorrect****
- Correct reference answer: IPPL sets up MPI in `src/Ippl.cpp::initialize()` by constructing `mpi::Environment` and `mpi::Communicator`; the low-level `MPI\_Init` check/call is in `src/Communicate/Environment.cpp`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.20`
- Reference identifier/location hits: `0/9`
- Latency: `9.59s`
- Matched reference signals: `sets, mpi`
- Missing/weak reference signals: `src/ippl.cpp::initialize, constructing, mpi::environment, mpi::communicator, low-level, mpi\_init, check/call, src/communicate/environment.cpp`
- Claude Code answer excerpt: `To determine where IPPL sets up the MPI (Message Passing Interface) environment, I'll search through the relevant files in the `data/raw/ippl` directory. The primary configuration and initialization of MPI environments are often found in scripts or configuration files that control the execution of parallel jobs. ###...`

loc_003 - Where is Kokkos initialized and finalized inside IPPL?

- Verdict: ****Partial****
- Correct reference answer: Kokkos is initialized and finalized directly in `src/Ippl.cpp` inside `ippl::initialize()` and `ippl::finalize()`.
- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `0/8`
- Latency: `5.79s`
- Matched reference signals: `kokkos, initialized, finalized`
- Missing/weak reference signals: `directly, src/ippl.cpp, inside, ippl::initialize, ippl::finalize, Ippl.cpp, finalize(), initialize()`
- Claude Code answer excerpt: `To find where Kokkos is initialized and finalized within the IPPL source tree, I'll search for relevant keywords like "Kokkos" in the `data/raw/ippl` directory. This will help me locate the specific files and functions where Kokkos is set up and torn down. I'll use the `ReadFile` tool to examine the contents of each...`

loc_004 - Where is the orthogonal recursive bisection load balancer implemented?

- Verdict: ****Incorrect****
- Correct reference answer: The ORB load balancer is implemented in ``src/Decomposition/OrthogonalRecursiveBisection.hpp`` with declarations in the matching header.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.29``
- Reference identifier/location hits: ``0/2``
- Latency: ``4.86s``
- Matched reference signals: ``load, balancer``
- Missing/weak reference signals: ``orb, src/decomposition/orthogonalrecursivebisection.hpp, declarations, matching, header, OrthogonalRecursiveBisection.hpp, src/Decomposition/OrthogonalRecursiveBisection.hpp``
- Claude Code answer excerpt: ``To find where the orthogonal recursive bisection load balancer is implemented in the IPPL source tree, we'll need to explore the relevant files and symbols within the codebase. Let's start by looking for any references to "orthogonal recursive bisection" or related terms. ### Step 1: Search for Relevant Terms First,...``

loc_005 - Where are boundary conditions applied to a Field?

- Verdict: ****Incorrect****
- Correct reference answer: Field BCs are attached in ``Field::setFieldBC()`` and actually applied through ``BConds::apply()`` and the concrete face classes in ``BcTypes.hpp``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.25``
- Reference identifier/location hits: ``0/7``
- Latency: ``7.98s``
- Matched reference signals: ``field, applied, classes``
- Missing/weak reference signals: ``bcs, attached, field::setfieldbc, actually, through, bconds::apply, concrete, face``
- Claude Code answer excerpt: ``To locate where boundary conditions are applied to a `Field`, we need to explore the source code in the IPPL repository. Specifically, we should look for files and symbols that define or apply boundary conditions. Let's start by searching for keywords related to boundary conditions in the IPPL source tree. We can us...``

loc_006 - Where is the CIC (cloud-in-cell) scatter implemented?

- Verdict: ****Partial****
- Correct reference answer: CIC scatter is implemented in ``src/Interpolation/CIC.hpp``, primarily ``scatterToPoint()`` and ``scatterToField()``.
- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: ``0.33``
- Reference identifier/location hits: ``0/4``
- Latency: ``9.09s``
- Matched reference signals: ``cic, scatter``
- Missing/weak reference signals: ``src/interpolation/cic.hpp, primarily, scattertopoint, scattertofield, CIC.hpp, scatterToField(), scatterToPoint(), src/Interpolation/CIC.hpp``
- Claude Code answer excerpt: ``The CIC (Cloud-In-Cell) scatter implementation can be found in several files within the IPPL source tree. Specifically, it is related to the implementation of cloud properties and their distribution across cells. Here are the key files and symbols you may want to explore: 1. **File: `src/ippl_core/field/cic_field.h`...``

loc_007 - Where is the CIC gather implemented?

- Verdict: ****Incorrect****
- Correct reference answer: CIC gather is implemented in ``src/Interpolation/CIC.hpp``, primarily ``gatherFromPoint()`` and ``gatherFromField()``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.00``
- Reference identifier/location hits: ``0/4``
- Latency: ``2.04s``
- Missing/weak reference signals: ``cic, gather, src/interpolation/cic.hpp, primarily, gatherfrompoint, gatherfromfield, CIC.hpp, gatherFromField()``
- Claude Code answer excerpt: ``To determine where the `CIC_gather` function is implemented in the codebase, I'll start by looking at the source files under `data/raw/ippl`. Given the structure described in the CLAUDE.md file, I'll focus on the ingestion stage to find relevant entities and their explanations. First, I'll use the `FileReader` tool...``

loc_008 - Where is halo exchange for BareField implemented?

- Verdict: ****Partial****
- Correct reference answer: Halo exchange for ``BareField`` is surfaced in ``src/Field/BareField.hpp`` and actually

performed in `src/Field/HaloCells.hpp`.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `1/5`
- Latency: `5.39s`
- Matched reference signals: `halo, exchange, barefield, BareField`
- Missing/weak reference signals: `surfaced, src/field/barefield.hpp, actually, performed, src/field/halocells.hpp, BareField.hpp, HaloCells.hpp, src/Field/BareField.hpp`
- Claude Code answer excerpt: `To locate the implementation of halo exchange for `BareField`, we need to explore the codebase under `data/raw/ippl`. Halo exchange typically involves exchanging data along the boundaries of a computational domain, often used in parallel computing or scientific simulations. Let's search for relevant keywords like "h..."`

loc_009 - Where are the FFT-based open-boundary Poisson solver routines defined?

- Verdict: ****Partial****
- Correct reference answer: The FFT open-boundary Poisson routines are defined in `src/PoissonSolvers/FFTOpenPoissonSolver.h` and `src/PoissonSolvers/FFTOpenPoissonSolver.hpp`.
- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: `0.67`
- Reference identifier/location hits: `0/4`
- Latency: `3.83s`
- Matched reference signals: `fft, open-boundary, poisson, defined`
- Missing/weak reference signals: `src/poissonsolvers/fftopenpoissonsolver.h, src/poissonsolvers/fftopenpoissonsolver.hpp, FFTOpenPoissonSolver.h, FFTOpenPoissonSolver.hpp, src/PoissonSolvers/FFTOpenPoissonSolver.h, src/PoissonSolvers/FFTOpenPoissonSolver.hpp`
- Claude Code answer excerpt: `To find where the FFT-based open-boundary Poisson solver routines are defined in the IPPL source tree, I will explore the repository files and documentation. 1. ****Search for relevant keywords****: - I'll start by searching through the codebase for keywords related to the FFT-based open-boundary Poisson solver. Common...`

loc_010 - Where is the conjugate gradient solver algorithm implemented?

- Verdict: ****Incorrect****
- Correct reference answer: The CG/PCG algorithm implementation is in `src/LinearSolvers/PCG.h`; the Poisson wrapper that instantiates it is `src/PoissonSolvers/PoissonCG.h`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.12`
- Reference identifier/location hits: `0/4`
- Latency: `1.95s`
- Matched reference signals: `algorithm`
- Missing/weak reference signals: `cg/pcg, implementation, src/linearsolvers/pcg.h, poisson, wrapper, instantiates, src/poissonsolvers/poissoncg.h, PCG.h`
- Claude Code answer excerpt: `To find where the conjugate gradient solver algorithm is implemented, we need to search through the repository for relevant code snippets. Let's use the `Search` tool to look for files that mention "conjugate gradient" or related keywords. ****Tool call:**** ``bash Search "conjugate gradient" `` Please provide the res...`

loc_011 - Where is the FEM Poisson solver implemented?

- Verdict: ****Partial****
- Correct reference answer: The FEM Poisson solver is implemented in `src/PoissonSolvers/FEMPoissonSolver.h`.
- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: `0.75`
- Reference identifier/location hits: `0/2`
- Latency: `6.82s`
- Matched reference signals: `fem, poisson, solver`
- Missing/weak reference signals: `src/poissonsolvers/fempoissonsolver.h, FEMPoissonSolver.h, src/PoissonSolvers/FEMPoissonSolver.h`
- Claude Code answer excerpt: `To find where the FEM Poisson solver is implemented in the IPPL source tree, we need to explore the codebase for relevant files and symbols. Given that the repository evidence may be insufficient, I'll outline a systematic approach to locate the implementation. **### Step 1: Search for Keywords** First, let's search for...`

loc_012 - Where is the FDTD Maxwell solver base class defined?

- Verdict: ****Partial****
- Correct reference answer: The FDTD Maxwell solver base class is defined in `src/MaxwellSolvers/FDTSolverBase.h` with implementation in `FDTSolverBase.hpp`.

- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: `0.62`
- Reference identifier/location hits: `0/3`
- Latency: `2.13s`
- Matched reference signals: `fdtd, maxwell, solver, base, defined`
- Missing/weak reference signals: `src/maxwellsolvers/fdtdsolverbase.h, implementation, fdtdsolverbase.hpp, FDTDSolverBase.h, FDTDSolverBase.hpp, src/MaxwellSolvers/FDTDSolverBase.h`
- Claude Code answer excerpt: `To find the location of the FDTD Maxwell solver base class, I will search through the codebase using the `grep` command to look for relevant keywords. This will help me pinpoint where the FDTD Maxwell solver base class is defined. ``sh grep -r "FDTD Maxwell solver base class" data/raw/ippl/ `` This command will re...`

loc_013 - Where is the LagrangeSpace class defined?

- Verdict: ****Partial****
- Correct reference answer: The `LagrangeSpace` class is defined in `src/FEM/LagrangeSpace.h` with methods implemented in `src/FEM/LagrangeSpace.hpp`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `1/5`
- Latency: `3.38s`
- Matched reference signals: `lagrangespace, defined, LagrangeSpace`
- Missing/weak reference signals: `src/fem/lagrangespace.h, src/fem/lagrangespace.hpp, LagrangeSpace.h, LagrangeSpace.hpp, src/FEM/LagrangeSpace.h, src/FEM/LagrangeSpace.hpp`
- Claude Code answer excerpt: `To find the `LagrangeSpace` class, I'll search through the codebase for its definition. Based on the repository structure and the files listed in the current context, I'll start by examining files that might contain definitions of mathematical or geometric classes. Let's check the following files: 1. **\*\*src/maths/geo...`**

loc_014 - Where is the particle update routine (that moves particles across ranks) implemented?

- Verdict: ****Partial****
- Correct reference answer: Particle migration between ranks is implemented in `ParticleSpatialLayout::update()` in `src/Particle/ParticleSpatialLayout.hpp`.
- Explanation: The answer is related, but it misses the concrete implementation location or symbol emphasized by the Codex reference.
- Reference-keyword coverage: `0.33`
- Reference identifier/location hits: `0/5`
- Latency: `5.51s`
- Matched reference signals: `particle, ranks`
- Missing/weak reference signals: `migration, between, particlespatiallayout::update, src/particle/particlespatiallayout.hpp, ParticleSpatialLayout.hpp, ParticleSpatialLayout::update, ParticleSpatialLayout::update(), src/Particle/ParticleSpatialLayout.hpp`
- Claude Code answer excerpt: `To find where the particle update routine is implemented in the IPPL source tree, I'll need to explore the codebase. The question asks about a routine that "moves particles across ranks," which suggests it's part of a parallel or distributed computing context. Given that the IPPL source tree is located under `data/r...`

loc_015 - Where are IPPL timers defined?

- Verdict: ****Partial****
- Correct reference answer: IPPL timers are defined by the singleton timing layer in `src/Utility/IpplTimings.h/.cpp`, not just the low-level `Timer` helper.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.36`
- Reference identifier/location hits: `1/3`
- Latency: `6.34s`
- Matched reference signals: `timers, defined, timing, timer, Timer`
- Missing/weak reference signals: `singleton, layer, src/utility/ippltimings.h/.cpp, not, just, low-level, helper, IpplTimings.h`
- Claude Code answer excerpt: `To find where IPPL timers are defined, we need to explore the source code in the `data/raw/ippl` directory. Given the question's context, I will search for files that might contain definitions or mentions of timers. **### Steps:** 1. **\*\*Search for timer-related keywords\*\*:** Words like "timer", "time", "clock", etc. 2. **\*\*C...`**

Class Responsibility

cls_001 - What is the main responsibility of the BareField class?

- Verdict: ****Correct****

- Correct reference answer: ``BareField`` is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.15``
- Reference identifier/location hits: ``1/1``
- Latency: ``4.39s``
- Matched reference signals: ``barefield, field, BareField``
- Missing/weak reference signals: ``core, distributed, storage, owns, view, domain, metadata, layout``
- Claude Code answer excerpt: ``To answer the question "What is the main responsibility of the BareField class?", I'll need to explore the codebase and understand what the `BareField` class does. First, let's locate the `BareField` class in the repository: 1. Search for the `BareField` class: - Use a search tool or command like `grep` to find...``

cls_002 - What is the main responsibility of the Field class compared to BareField?

- Verdict: ****Correct****
- Correct reference answer: ``Field`` extends ``BareField`` with a mesh, boundary-condition container, and mesh-aware integral/average semantics.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.33``
- Reference identifier/location hits: ``2/2``
- Latency: ``4.91s``
- Matched reference signals: ``field, extends, barefield, BareField, Field``
- Missing/weak reference signals: ``mesh, boundary-condition, container, mesh-aware, integral/average, semantics``
- Claude Code answer excerpt: ``To determine the main responsibility of the `Field` class compared to `BareField`, I'll analyze the relevant code in `data/raw/ippl`. 1. Locate the classes: - `Field`: This is likely a custom field type that extends or wraps another field. - `BareField`: This appears to be a base field type. 2. Explore `Field`...``

cls_003 - What does the FieldLayout class do and what are its main template parameters?

- Verdict: ****Incorrect****
- Correct reference answer: ``FieldLayout<Dim>`` distributes a global ``NDIndex<Dim>`` over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is ``Dim``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.12``
- Reference identifier/location hits: ``0/3``
- Latency: ``1.77s``
- Matched reference signals: ``fieldlayout, template``
- Missing/weak reference signals: ``dim, distributes, global, ndindex, ranks, stores, local, domains``
- Claude Code answer excerpt: ``To answer the question about the `FieldLayout` class and its main template parameters, I'll need to explore the source code. The `FieldLayout` class is likely found in one of the header files under `data/raw/ippl`. Let's start by searching for `FieldLayout` in the repository using the search tool. ### Action: Search...``

cls_004 - What is the purpose of the ParticleBase class and how is it meant to be used?

- Verdict: ****Correct****
- Correct reference answer: ``ParticleBase`` is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.29``
- Reference identifier/location hits: ``1/1``
- Latency: ``7.45s``
- Matched reference signals: ``particlebase, base, particles, attributes, ParticleBase``
- Missing/weak reference signals: ``user-extensible, container, derived, classes, register, them, layout-driven, migration``
- Claude Code answer excerpt: ``The `ParticleBase` class appears to be a foundational class in an IPPL (Inertial Propulsion Propulsion Laboratory) benchmark. Based on the available files and code structure, here's what I can determine about its purpose and intended use: ### Purpose of ParticleBase Class 1. Abstract Base Class: The `ParticleBas...``

cls_005 - What does the ParticleSpatialLayout class do?

- Verdict: ****Partial****
- Correct reference answer: ``ParticleSpatialLayout`` decides which spatial region/rank owns each particle and performs the migration/update workflow after particles move.
- Explanation: The answer contains some relevant information but also indicates uncertainty or inability to

fully answer.

- Reference-keyword coverage: `0.31`
- Reference identifier/location hits: `1/1`
- Latency: `7.85s`
- Matched reference signals: `particlespatiallayout, spatial, particle, particles, ParticleSpatialLayout`
- Missing/weak reference signals: `decides, region/rank, owns, each, performs, migration/update, workflow, after`
- Claude Code answer excerpt: `The `ParticleSpatialLayout` class is not directly present in the provided repository files under `data/raw/ippl`. However, based on the context and files available, it's likely that this class is part of a larger system for simulating particle dynamics. Let's explore relevant files to understand its potential role....`

cls_006 - What is the responsibility of UniformCartesian?

- Verdict: ****Correct****
- Correct reference answer: `UniformCartesian` represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.30`
- Reference identifier/location hits: `1/1`
- Latency: `6.09s`
- Matched reference signals: `uniformcartesian, uniform, cartesian, UniformCartesian`
- Missing/weak reference signals: `represents, mesh, exposes, spacing/origin/volume, vertex-position, geometry, queries`
- Claude Code answer excerpt: `To determine the responsibility of `UniformCartesian`, let's first look at its definition and any related code in the IPPL repository. ### Step 1: Locate `UniformCartesian` The code for `UniformCartesian` can be found in the file `data/raw/ippl/src/geometry/cartesian.py`. ``python # data/raw/ippl/src/geometry/carte...`

cls_007 - What does the Mesh base class define?

- Verdict: ****Correct****
- Correct reference answer: `Mesh` is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.15`
- Reference identifier/location hits: `1/1`
- Latency: `4.81s`
- Matched reference signals: `mesh, base, Mesh`
- Missing/weak reference signals: `abstract, interface, meshes, origin, grid, size, spacing, vertex`
- Claude Code answer excerpt: `To answer this question, I'll need to explore the codebase under `data/raw/ippl` and identify where the `Mesh` base class is defined. The `Mesh` class should be part of the IPPL library, which is not checked into the repository but is expected to be located in the `data/raw/ippl` directory. Let's start by searching...`

cls_008 - What is the responsibility of the Communicator class in ippl::mpi?

- Verdict: ****Partial****
- Correct reference answer: `ippl::mpi::Communicator` wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.30`
- Reference identifier/location hits: `0/1`
- Latency: `7.53s`
- Matched reference signals: `mpi, tag, communication`
- Missing/weak reference signals: `ippl::mpi::communicator, wraps, point-to-point, collectives, handling, managed, buffers, ippl::mpi::Communicator`
- Claude Code answer excerpt: `To answer this question, I'll need to search through the source code under `data/raw/ippl` for any mention of the `Communicator` class and determine its responsibilities. Let's start by searching for the class definition. ### Searching for `Communicator` I'll use a text-based search tool to look for the `Communicato...`

cls_009 - What is the role of the Environment class in ippl::mpi?

- Verdict: ****Incorrect****
- Correct reference answer: `ippl::mpi::Environment` manages MPI environment lifetime checks/teardown and provides an abort hook.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.12`

- Reference identifier/location hits: `0/1`
- Latency: `5.60s`
- Matched reference signals: `environment`
- Missing/weak reference signals: `ippl::mpi::environment, manages, mpi, lifetime, checks/teardown, abort, hook, ippl::mpi::Environment`
- Claude Code answer excerpt: `To answer this question, we need to explore the source code and documentation related to the IPPL library. The `Environment` class is likely defined in one of the header files under the `ippl/mpi` directory. Let's start by navigating to the relevant files using our read-only tools: 1. **FileReader**: This tool will...`

cls_010 - What does the Archive class do in the Communicate module?

- Verdict: **Correct**
- Correct reference answer: `Archive` serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.08`
- Reference identifier/location hits: `1/1`
- Latency: `4.10s`
- Matched reference signals: `archive, Archive`
- Missing/weak reference signals: `serializes, deserializes, kokkos, views, vector-valued, raw, byte, buffers`
- Claude Code answer excerpt: `The `Archive` class within the `Communicate` module appears to be responsible for managing and storing communication logs or records. It likely provides functionality to save, retrieve, and possibly purge communication data. To provide a more detailed answer, I would need to explore the code in the `data/raw/ippl` d...`

cls_011 - What is the role of the BConds class?

- Verdict: **Correct**
- Correct reference answer: `BConds` is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.10`
- Reference identifier/location hits: `1/1`
- Latency: `4.10s`
- Matched reference signals: `bconds, BConds`
- Missing/weak reference signals: `container, boundary-condition, faces, orchestrates, neighbor, discovery, application, ghost-to-physical`
- Claude Code answer excerpt: `To determine the role of the `BConds` class, we need to explore its definition and usage within the repository. Let's start by finding where `BConds` is defined. After reviewing the codebase, I found that `BConds` is defined in the file `src/ingestion/explanation_generator.py`. Here is the relevant part of the code:...`

cls_012 - What is the role of the BaseManager class?

- Verdict: **Partial**
- Correct reference answer: `BaseManager` provides the simulation loop skeleton: `pre\_run`, `pre\_step`, `advance`, `post\_step`, and `run(nt)`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.11`
- Reference identifier/location hits: `1/6`
- Latency: `12.28s`
- Matched reference signals: `basemanager, BaseManager`
- Missing/weak reference signals: `simulation, loop, skeleton, pre\_run, pre\_step, advance, post\_step, run`
- Claude Code answer excerpt: `To understand the role of the `BaseManager` class in IPPL (Intelligent Process for Programming Language), we need to explore its definition and usage within the codebase. **Step 1:** Locate the `BaseManager` Class First, I will search for the `BaseManager` class in the repository files under `data/raw/ippl`. **Searc...**`

cls_013 - What does the Index class represent and what operations does it support?

- Verdict: **Correct**
- Correct reference answer: `Index` represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.23`
- Reference identifier/location hits: `1/1`
- Latency: `4.46s`
- Matched reference signals: `index, supports, operations, Index`

- Missing/weak reference signals: `represents, regular, strided, integer, range/slice, range-style, arithmetic, comparison`
- Claude Code answer excerpt: `The `Index` class appears to be a central component of the system, likely responsible for managing the indexing and retrieval of code entities. Based on the context provided, here are some key points about the `Index` class: 1. **Responsibility**: The `Index` class is responsible for indexing and retrieving code ent...`

Algorithm

alg_001 - How does the FFT-based open-boundary Poisson solver compute the electrostatic potential?

- Verdict: **Partial**
- Correct reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.33`
- Latency: `5.59s`
- Matched reference signals: `fft, open-boundary, solver, ffts, fourier, inverse, computes`
- Missing/weak reference signals: `embeds/extends, rhs, onto, larger, domains, both, density, precomputed`
- Claude Code answer excerpt: `To answer the question about how the FFT-based open-boundary Poisson solver computes the electrostatic potential, we need to delve into the relevant source files and understand their interplay. Based on the repository structure and documentation, here's a step-by-step breakdown of the process:
1. **Data Preparat...**`

alg_002 - How does the FFT-based periodic Poisson solver work in IPPL?

- Verdict: **Partial**
- Correct reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by $|k|^2$ (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by $ik/|k|^2$ before the inverse FFT.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `0/2`
- Latency: `8.41s`
- Matched reference signals: `periodic, fft, solver, ffts, fourier, then, inverse`
- Missing/weak reference signals: `charge, density, divides, each, mode, zero, handled, separately`
- Claude Code answer excerpt: `To understand how the FFT-based periodic Poisson solver works in IPPL, let's explore relevant parts of the codebase. The primary file to focus on is likely `src/fft_poisson_solver.py`, which contains the implementation of the solver. **Key Components** 1. **File: src/fft\_poisson\_solver.py** - This file is crucial as...`

alg_003 - How does the truncated Green periodic Poisson solver differ from the standard periodic one?

- Verdict: **Incorrect**
- Correct reference answer: The truncated-Green periodic solver replaces the simple $1/|k|^2$ kernel with the FFT of a truncated/erf-smoothed Green's function $\text{forceConstant} * \text{erf}(\alpha r) / r$, then follows the same transform-multiply-inverse pattern.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.18`
- Reference identifier/location hits: `0/3`
- Latency: `7.96s`
- Matched reference signals: `periodic, solver, green`
- Missing/weak reference signals: `truncated-green, replaces, simple, kernel, fft, truncated/erf-smoothed, forceconstant, erf`
- Claude Code answer excerpt: `To answer how the truncated Green periodic Poisson solver differs from the standard periodic one, we need to explore the relevant files and symbols in the repository. Let's start by identifying where these solvers are implemented. **Step 1: Identify Relevant Files** First, let's find the files that contain implement...`

alg_004 - How does IPPL's CG solver iterate in the PCG.h implementation?

- Verdict: **Correct**
- Correct reference answer: In `PCG.h`, the CG iteration forms residual/direction fields, applies the operator, computes `alpha` and `beta` from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.39`
- Reference identifier/location hits: `3/4`

- Latency: `7.12s`
- Matched reference signals: `pcg.h, iteration, computes, alpha, beta, tolerance, preconditioned, residual`
- Missing/weak reference signals: `forms, residual/direction, fields, applies, operator, inner, products, updates`
- Claude Code answer excerpt: `To answer how IPPL's CG solver iterates in the `PCG.h` implementation, we need to explore the relevant files and symbols in the repository. Specifically, we'll look at the `PCG` class and its methods. Here is a summary of how the PCG (Preconditioned Conjugate Gradient) solver iterates according to the code: 1. **Ini...**`

alg_005 - How is cloud-in-cell interpolation performed in IPPL (both scatter and gather directions)

- Verdict: ****Incorrect****
- Correct reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the 2^{Dim} neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.11`
- Reference identifier/location hits: `0/1`
- Latency: `6.63s`
- Matched reference signals: `grid, particles`
- Missing/weak reference signals: `cic, computes, lower/upper, weights, per, axis, scatters, atomic`
- Claude Code answer excerpt: `To answer the question about cloud-in-cell interpolation in IPPL, we need to look into the source code. The key components of cloud-in-cell interpolation are typically found in files that handle grid operations and particle distribution. Let's start by examining the `FileReader` class, which is responsible for loadi...`

alg_006 - How does the standard FDTD solver advance the electromagnetic field in time?

- Verdict: ****Correct****
- Correct reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.45`
- Latency: `7.09s`
- Matched reference signals: `standard, fdtd, advances, field, finite-difference, update, previous, current`
- Missing/weak reference signals: `solver, source-like, second-order, levels, cells, source, terms, then`
- Claude Code answer excerpt: `The standard Finite-Difference Time-Domain (FDTD) method advances the electromagnetic field in time by solving Maxwell's equations discretized in both space and time. This process involves updating the electric and magnetic fields at each grid point based on their values at the previous time step. To understand how...`

alg_007 - How does the LagrangeSpace class evaluate degrees of freedom per element?

- Verdict: ****Correct****
- Correct reference answer: `LagrangeSpace` evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.40`
- Reference identifier/location hits: `1/1`
- Latency: `8.72s`
- Matched reference signals: `lagrangespace, evaluates, element, dofs, then, LagrangeSpace`
- Missing/weak reference signals: `mapping, local, indices, global, dof, order-1, reference-element, shape`
- Claude Code answer excerpt: `To understand how the `LagrangeSpace` class evaluates degrees of freedom (DOFs) per element, we need to examine its implementation. The `LagrangeSpace` class is likely part of a larger codebase that deals with finite element analysis or similar numerical simulations. Let's start by exploring the relevant files and s...`

alg_008 - How is Gauss-Jacobi quadrature implemented and used in IPPL FEM?

- Verdict: ****Partial****
- Correct reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrsFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.22`
- Latency: `10.25s`
- Matched reference signals: `gauss-jacobi, quadrature, computes, fem`
- Missing/weak reference signals: `nodes/weights, newton, iteration, jacobi, polynomial, roots, starting, chebyshev`

- Claude Code answer excerpt: `To answer the question about how Gauss-Jacobi quadrature is implemented and used in IPPL FEM, I'll need to look for relevant files and symbols in the repository. Let's start by identifying potential candidates based on the question. ### Step 1: Identify Relevant Files Gauss-Jacobi quadrature is typically associated...`

alg_009 - How does the orthogonal recursive bisection decide where to cut the domain?

- Verdict: ****Partial****
- Correct reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.31`
- Latency: `6.11s`
- Matched reference signals: `cut, domain, then, recursive, more`
- Missing/weak reference signals: `orb, chooses, axis, geometry, finds, weight-balanced, cut/median, splits`
- Claude Code answer excerpt: `To answer the question "How does the orthogonal recursive bisection decide where to cut the domain?" in the context of IPPL, we need to explore the relevant files and symbols within the codebase. Let's break down the steps: 1. ****Identify Relevant Files****: We need to locate files that mention `orthogonal recursive bi...`

alg_010 - How does inverse transform sampling generate particles from a given distribution in IPPL?

- Verdict: ****Correct****
- Correct reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.56`
- Latency: `10.14s`
- Matched reference signals: `inverse, transform, sampling, maps, uniform, random, cdf, samples`
- Missing/weak reference signals: `values, space, estimating, refining, newton-raphson, dimension`
- Claude Code answer excerpt: `To understand how inverse transform sampling generates particles from a given distribution in IPPL (Inference Parameterized Particle Library), let's explore the relevant code and documentation. The key files to look at are: 1. **\*\*File Path\*\***: `data/raw/ippl/src/alg_010.cpp` - This file contains the implementation of...`

alg_011 - How does the FFT class interface with heFFTe to perform complex-to-complex and real-to-complex transforms?

- Verdict: ****Incorrect****
- Correct reference answer: The FFT layer builds heFFTe boxes/plans in `setup()`, copies IPPL field data into the required views/subviews, calls heFFTe forward/backward transforms, and copies results back for CC/RC/DST/DCT variants.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.21`
- Reference identifier/location hits: `0/1`
- Latency: `7.13s`
- Matched reference signals: `fft, heffte, transforms`
- Missing/weak reference signals: `layer, builds, boxes/plans, setup, copies, field, required, views/subviews`
- Claude Code answer excerpt: `To answer how the FFT class interfaces with heFFTe for performing complex-to-complex and real-to-complex transforms, we need to look into the source code of the IPPL library. Specifically, we should examine the files that define the FFT class and its interaction with heFFTe. Let's start by looking at the `fft.hpp` f...`

alg_012 - How does IPPL's preconditioned CG use the preconditioner defined in Preconditioner.h?

- Verdict: ****Correct****
- Correct reference answer: PCG applies a chosen preconditioner from `Preconditioner.h` to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.19`
- Reference identifier/location hits: `1/1`
- Latency: `5.75s`
- Matched reference signals: `preconditioner, preconditioner.h, preconditioned, Preconditioner.h`
- Missing/weak reference signals: `pcg, applies, chosen, residual, inner, products, direction, updates`
- Claude Code answer excerpt: `To answer this question, we need to investigate how IPPL's preconditioned conjugate gradient (CG) method uses the preconditioner defined in `Preconditioner.h`. Let's start by exploring the relevant files and symbols. ### 1. Identify Relevant Files First, let's locate the file where the preconditioned CG method is im...`

Data Flow

flow_001 - How does charge density flow from particles to the grid in a typical PIC step in IPPL?

- Verdict: ****Incorrect****

- Correct reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (`scatter`) or FEM assembly (`assemble\_rhs\_from\_particles`), accumulated into `rho`, and then normalized/background-shifted in the manager before solving.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.20`

- Reference identifier/location hits: `0/3`

- Latency: `6.74s`

- Matched reference signals: `pic, particle, positions, grid`

- Missing/weak reference signals: `workflow, charges, deposited, onto, cic, scatter, fem, assembly`

- Claude Code answer excerpt: `To answer the question about how charge density flows from particles to the grid in a typical PIC (Particle-In-Cell) simulation, we need to explore the relevant code and documentation in the IPPL repository. ### Key Concepts: 1. **Charge Density**: A measure of the total electric charge distributed within a given vo...`

flow_002 - How does the electric field flow from the grid back to particles after a Poisson solve?

- Verdict: ****Correct****

- Correct reference answer: After the field solve, the manager gathers the electric field back to particles with CIC `gather(...)` or FEM `interpolate\_grad\_to\_diracs(...)`, storing particle-local `E` values for the push step.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.50`

- Reference identifier/location hits: `0/2`

- Latency: `10.54s`

- Matched reference signals: `after, field, solve, electric, back, particles, values, step`

- Missing/weak reference signals: `manager, gathers, cic, gather, fem, interpolate\_grad\_to\_diracs, storing, particle-local`

- Claude Code answer excerpt: `To answer this question, we need to explore how the electric field flows from the grid back to particles in an IPPL simulation. This involves understanding the data flow within the IPPL codebase. ### Step 1: Identify Relevant Files and Symbols We'll start by identifying the key files and symbols that handle the data...`

flow_003 - How are halo cells exchanged between neighboring MPI ranks for a BareField?

- Verdict: ****Incorrect****

- Correct reference answer: Halo exchange uses `FieldLayout` neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.21`

- Reference identifier/location hits: `0/1`

- Latency: `5.26s`

- Matched reference signals: `halo, exchange, mpi, handle`

- Missing/weak reference signals: `fieldlayout, neighbor/range, metadata, pack, field, subviews, send/receive, them`

- Claude Code answer excerpt: `To answer this question about how halo cells are exchanged between neighboring MPI ranks in a BareField simulation, we need to understand the relevant code and configuration files. The IPPL (Intelligent Parallel Particle Library) codebase is likely to contain the necessary information. Here are some steps to explore...`

flow_004 - How are particles migrated between MPI ranks by ParticleSpatialLayout?

- Verdict: ****Incorrect****

- Correct reference answer: `ParticleSpatialLayout::update()` applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.

- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.

- Reference-keyword coverage: `0.18`

- Reference identifier/location hits: `0/3`

- Latency: `5.13s`

- Matched reference signals: `particle, ranks, all, particles`

- Missing/weak reference signals: `particlespatiallayout::update, applies, bcs, locates, destination, advertises, receive, counts`

- Claude Code answer excerpt: `To understand how particles are migrated between MPI ranks by `ParticleSpatialLayout`, we need to explore the relevant code in the IPPL source tree. Based on the provided context, the relevant files and symbols are likely found in: 1. **File:** `src/Pixel/ParticleSpatialLayout.cpp` 2. **Symbols:** Functions related...`

flow_005 - How does FieldLayout decide which index ranges live on which rank?

- Verdict: **Partial**

- Correct reference answer: `FieldLayout` uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local `NDIndex`, plus neighbor and communication range tables.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.27`

- Reference identifier/location hits: `1/2`

- Latency: `10.98s`

- Matched reference signals: `fieldlayout, each, rank, range, FieldLayout`

- Missing/weak reference signals: `partitioner, requested, parallel/serial, decomposition, flags, assign, local, ndindex`

- Claude Code answer excerpt: `To determine how `FieldLayout` decides which index ranges live on which rank, we need to explore the relevant code and documentation in the IPPL source tree. Based on the provided context, I will proceed with the following steps: 1. **Locate the `FieldLayout` class:** - The `FieldLayout` class is likely defined some...`

flow_006 - How is a ParameterList used to configure the FFT and solver classes?

- Verdict: **Partial**

- Correct reference answer: `ParameterList` is merged into FFT and solver objects, then consumed by `setup()/initialize()/solve()` for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.26`

- Reference identifier/location hits: `1/5`

- Latency: `6.80s`

- Matched reference signals: `parameterlist, fft, solver, then, initialize, ParameterList`

- Missing/weak reference signals: `merged, objects, consumed, setup, solve, communication, mode, pencil/reorder`

- Claude Code answer excerpt: `To answer how a `ParameterList` is used to configure the FFT and solver classes in an IPPL benchmark question, let's explore the relevant files and symbols. **Relevant Files** 1. **File:** `data/raw/ippl/include/fftw3.h` - This file contains the declarations for the FFTW library, which is likely used for performing...`

flow_007 - How does data flow through the BaseManager's pre_step / advance / post_step hooks during simulation?

- Verdict: **Partial**

- Correct reference answer: `BaseManager::run()` executes `pre\_step()`, `advance()`, and `post\_step()` for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.31`

- Reference identifier/location hits: `0/6`

- Latency: `5.70s`

- Matched reference signals: `pre\_step, advance, post\_step, each, hooks`

- Missing/weak reference signals: `basemanager::run, executes, timestep, derived, managers, use, those, deposit/solve/gather`

- Claude Code answer excerpt: `To answer how data flows through the `BaseManager`'s `pre_step`, `advance`, and `post_step` hooks during a simulation, we need to examine the relevant parts of the codebase. Based on the repository structure and documentation provided, I will guide you through the process. 1. **Locating Relevant Files:** - The BaseM...`

flow_008 - How does the FEM solver move data between fields and FEMVector representations?

- Verdict: **Correct**

- Correct reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (`evaluateLoadVector`, `evaluateAx`, `FEMVector`) and then interpolating the solved field quantities back where needed.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.44`

- Reference identifier/location hits: `1/3`

- Latency: `8.08s`

- Matched reference signals: `fem, move, field, representations, femvector, quantities, FEMVector`

- Missing/weak reference signals: `solvers, assembling, load/stiffness, evaluateloadvector, evaluateax, then, interpolating, solved`
- Claude Code answer excerpt: `To answer the question "How does the FEM solver move data between fields and FEMVector representations?", let's examine the relevant code in the `data/raw/ippl` directory. The primary components involved are: 1. **Fields**: Represent different physical quantities like displacement, stress, etc. 2. **FEMVectors**: Re...`

flow_009 - How does OrthogonalRecursiveBisection redistribute particles after rebalancing?

- Verdict: **Partial**
- Correct reference answer: After ORB recomputes the decomposition, particle ownership changes are realized by updating the layout/regions and then migrating particles with the usual particle update path.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.29`
- Latency: `9.31s`
- Matched reference signals: `after, particle, then, particles`
- Missing/weak reference signals: `orb, recomputes, decomposition, ownership, changes, realized, updating, layout/regions`
- Claude Code answer excerpt: `To answer the question about how `OrthogonalRecursiveBisection` redistributes particles after rebalancing, I'll need to explore the relevant code in the IPPL source tree. First, let's identify where `OrthogonalRecursiveBisection` might be defined. Since we don't have direct access to the IPPL source tree here, we ne...`

Api Usage

api_001 - How do I create a 3D Field of doubles in IPPL?

- Verdict: **Partial**
- Correct reference answer: A typical 3D field is `Field<double, 3, UniformCartesian<double, 3>, Cell>` constructed from a `UniformCartesian` mesh and a `FieldLayout<3>`.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.36`
- Reference identifier/location hits: `0/3`
- Latency: `7.41s`
- Matched reference signals: `field, double`
- Missing/weak reference signals: `typical, uniformcartesian, cell, constructed, mesh, fieldlayout, Field<double, 3, UniformCartesian<double, 3>, Cell>, FieldLayout<3>`
- Claude Code answer excerpt: `To create a 3D field of doubles in IPPL (assuming it's some kind of scientific computing or simulation framework), you'll need to look into how data structures and arrays are handled in the IPPL source code. Here's a step-by-step approach to understand and implement this: 1. **Identify Relevant Files**: - Look for f...`

api_002 - How do I construct a UniformCartesian mesh with custom spacing and origin?

- Verdict: **Correct**
- Correct reference answer: Construct the mesh as `UniformCartesian<double, Dim> mesh(domain, hx, origin);` where `hx` is the spacing vector and `origin` is the mesh origin.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.77`
- Reference identifier/location hits: `1/4`
- Latency: `6.56s`
- Matched reference signals: `construct, mesh, uniformcartesian, double, origin, spacing`
- Missing/weak reference signals: `dim, domain, vector, UniformCartesian<double, Dim> mesh(domain, hx, origin), hx, mesh(domain, hx, origin)`
- Claude Code answer excerpt: `To construct a `UniformCartesian` mesh with custom spacing and origin in IPPL (In situ Performance Prediction Library), you can follow the steps below. This example assumes that IPPL's source tree is properly cloned into `data/raw/ippl`. **Step 1: Import Necessary Headers** First, include the necessary headers for w...`

api_003 - How do I attach a boundary condition (e.g. periodic) to a Field?

- Verdict: **Incorrect**
- Correct reference answer: Create a `BConds<FieldT, Dim>`, assign per-face BC objects such as `PeriodicFace<FieldT>`, then call `field.setFieldBC(bc)`.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.23`
- Reference identifier/location hits: `0/4`
- Latency: `3.85s`
- Matched reference signals: `create, such, call`

- Missing/weak reference signals: `bconds, fieldt, dim, assign, per-face, objects, periodicface, then`
- Claude Code answer excerpt: `To attach a boundary condition such as periodic to a field in the IPPL source tree, you will need to modify the relevant files and add the necessary code for the boundary condition. Here's an example of how you can do this: 1. **\*\*Locate the Field Class\*\***: The `Field` class is likely defined in one of the files under...`

api_004 - How do I define a custom particle class that adds mass and velocity attributes on top of ParticleBase?

- Verdict: ****Correct****
- Correct reference answer: Define a class derived from `ParticleBase<ParticleSpatialLayout<...>>`, add `mass` and `velocity` attributes as members, and register them with `addAttribute()` in the constructor.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.50`
- Reference identifier/location hits: `2/4`
- Latency: `5.62s`
- Matched reference signals: `define, particlebase, mass, velocity, attributes, constructor`
- Missing/weak reference signals: `derived, particlespatiallayout, members, register, them, addattribute, ParticleBase<ParticleSpatialLayout<...>>, addAttribute()`
- Claude Code answer excerpt: `To define a custom particle class in IPPL that adds mass and velocity attributes on top of `ParticleBase`, you can extend the `ParticleBase` class and add the additional attributes. Here's how you can do it: 1. **\*\*Create or Open an Appropriate File\*\***: You might create a new file specifically for your custom particle...`

api_005 - How do I call ippl::initialize and ippl::finalize in a main program?

- Verdict: ****Correct****
- Correct reference answer: Call the free namespace functions `ippl::initialize(argc, argv);` and `ippl::finalize();` from `main`, as shown in the docs/examples.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.70`
- Reference identifier/location hits: `5/7`
- Latency: `3.21s`
- Matched reference signals: `call, namespace, ippl::initialize, argc, argv, ippl::finalize, main, finalize()`
- Missing/weak reference signals: `free, shown, docs/examples, initialize(argc, argv), ippl::initialize(argc, argv)`
- Claude Code answer excerpt: `To call `ippl::initialize` and `ippl::finalize` in a main program, you need to include the appropriate headers and ensure that these functions are called at the beginning and end of your program respectively. Here's an example of how you can do this: 1. Include the necessary header file: ``cpp
#include "IPPL/initia...`

api_006 - How do I set up and run a forward FFT on an IPPL Field?

- Verdict: ****Partial****
- Correct reference answer: Set up the relevant layouts/fields/parameters, build an `FFT<CCTransform, ...>` or `FFT<RCTransform, ...>` object, and call `transform(FORWARD, ...)` on the field(s).
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `0/4`
- Latency: `7.90s`
- Matched reference signals: `fft, transform, forward, field`
- Missing/weak reference signals: `set, relevant, layouts/fields/parameters, build, cctransform, rctransform, object, call`
- Claude Code answer excerpt: `To perform a forward Fast Fourier Transform (FFT) on an IPPL `Field`, you'll need to follow these steps: 1. **\*\*Include the necessary headers\*\***: Ensure that your source file includes the headers for IPPL and any other libraries required for FFT operations. 2. **\*\*Create or load an IPPL Field\*\***: Depending on whether you...`

api_007 - How do I construct an FFTPeriodicPoissonSolver and solve for the potential?

- Verdict: ****Correct****
- Correct reference answer: Construct `FFTPeriodicPoissonSolver(lhs, rhs)` (or default construct then set fields/params), ensure the RHS is attached, and call `solve()` to write the potential/gradient outputs.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.41`
- Reference identifier/location hits: `1/2`
- Latency: `6.95s`
- Matched reference signals: `construct, fftperiodicpoissonsolver, set, ensure, call, solve, solve()`
- Missing/weak reference signals: `lhs, rhs, default, then, fields/params, attached, write, potential/gradient`

- Claude Code answer excerpt: `To construct an `FFTPeriodicPoissonSolver` and solve for the potential in IPPL, you will need to follow several steps. Below is a detailed guide based on the codebase and documentation provided. ### Step-by-Step Guide 1. \*\*Import Necessary Modules\*\*: Ensure that all necessary modules are imported. Typically, this in...`

api_008 - How do I invoke the conjugate gradient Poisson solver on a Field?

- Verdict: **Partial**

- Correct reference answer: Use `PoissonCG<FieldT>` (or the Alpine `FieldSolver` wrapper around it), set its fields/parameters, and call `solve()` so the internal CG/PCG object iterates on the field.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.25`

- Reference identifier/location hits: `0/3`

- Latency: `6.97s`

- Matched reference signals: `use, set, object, field`

- Missing/weak reference signals: `poissoncg, fieldt, alpine, fieldsolver, wrapper, around, fields/parameters, call`

- Claude Code answer excerpt: `To invoke the conjugate gradient Poisson solver on a Field, you need to follow these steps: 1. **Understand the Context**: The IPPL (Incompressible Particle-in-Cell) benchmark question asks how to use a specific solver within the IPPL codebase. 2. **Identify Relevant Code**: You need to locate the relevant code that...`

api_009 - How do I sample particles from a normal distribution using IPPL Random utilities?

- Verdict: **Partial**

- Correct reference answer: For Gaussian draws, IPPL exposes both `randn` convenience generation and the more general `InverseTransformSampling` + `NormalDistribution` path.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: `0.08`

- Reference identifier/location hits: `1/3`

- Latency: `4.94s`

- Matched reference signals: `more, NormalDistribution`

- Missing/weak reference signals: `gaussian, draws, exposes, both, randn, convenience, generation, general`

- Claude Code answer excerpt: `To sample particles from a normal distribution in IPPL, you can use the `normal_distribution` class and its associated methods. Below is an example of how to use these classes: 1. **Include the necessary header file**: ``cpp #include "ippl/IPPLConfig.h" #include "ippl/random/Random.h" `` 2. **Create a random numbe...`

api_010 - How do I register particle attributes with ParticleBase so that they are communicated on migration?

- Verdict: **Correct**

- Correct reference answer: Register each particle attribute with `addAttribute(...)`; migration then serializes and transfers all registered attributes automatically.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.54`

- Reference identifier/location hits: `0/1`

- Latency: `6.39s`

- Matched reference signals: `register, particle, attribute, migration, serializes, registered, attributes`

- Missing/weak reference signals: `each, addattribute, then, transfers, all, automatically, addAttribute(...)`

- Claude Code answer excerpt: `To register particle attributes with `ParticleBase` so that they are communicated during migration, you need to follow these steps: 1. \*\*Define the Attributes\*\*: Identify and define the attributes that need to be registered. 2. \*\*Implement Attribute Registration\*\*: Implement a method in your class to register these...`

api_011 - How do I access the MPI rank and size through the IPPL Communicator?

- Verdict: **Correct**

- Correct reference answer: Access rank and size with `ippl::Comm->rank()` and `ippl::Comm->size()` (or the same methods on a `Communicator` object).

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: `0.60`

- Reference identifier/location hits: `3/6`

- Latency: `5.30s`

- Matched reference signals: `access, rank, size, communicator, Communicator, rank(), size()`

- Missing/weak reference signals: `ippl::comm-, same, object, ippl::Comm, ippl::Comm->rank(), ippl::Comm->size()`

- Claude Code answer excerpt: `To access the MPI (Message Passing Interface) rank and size through the IPPL Communicator, you need to use specific functions provided by the MPI library. The `Communicator` class in IPPL

is responsible for managing communication between processes. Here's a step-by-step guide on how to achieve this:

1. `**Include the...`

api_012 - How do I use IpplTimings to measure a region of code?

- Verdict: `**Incorrect**`
- Correct reference answer: Get a timer handle with ``IpplTimings::getTimer("name")``, surround the code with ``startTimer()`` / ``stopTimer()``, and print with ``IpplTimings::print()``.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: ``0.20``
- Reference identifier/location hits: ``0/8``
- Latency: ``4.26s``
- Matched reference signals: ``timer, name``
- Missing/weak reference signals: ``get, handle, ippltimings::gettimer, surround, starttimer, stoptimer, print, ippltimings::print``
- Claude Code answer excerpt: ``To measure a region of code using `IpplTimings`, you need to follow these steps: 1. **Import the necessary module**: Ensure that you have imported the `IpplTimings` class from the appropriate module. 2. **Create an instance of `IpplTimings`**: Instantiate the `IpplTimings` class before the region of code you want to...`

Parallelism And Kokkos

par_001 - How does IPPL achieve performance portability across CPUs and GPUs?

- Verdict: `**Correct**`
- Correct reference answer: IPPL gets performance portability by combining Kokkos execution/memory abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.50``
- Latency: ``9.14s``
- Matched reference signals: ``performance, portability, kokkos, mpi, target, cpus, gpus``
- Missing/weak reference signals: ``gets, combining, execution/memory, abstractions, heffte, same, paths``
- Claude Code answer excerpt: ``To understand how IPPL achieves performance portability across CPUs and GPUs, we need to explore the codebase and documentation in detail. Specifically, we should look at how IPPL is configured for different architectures and how it leverages parallel computing frameworks. ### Steps to Investigate: 1. **Check Config...`

par_002 - Why does IPPL use Kokkos::View for field storage?

- Verdict: `**Correct**`
- Correct reference answer: IPPL uses ``Kokkos::View`` because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: ``0.43``
- Reference identifier/location hits: ``1/1``
- Latency: ``5.47s``
- Matched reference signals: ``kokkos::view, because, need, storage, memory, spaces, Kokkos::View``
- Missing/weak reference signals: ``fields, execution-space-aware, subviews, mirrors, portable, kernels, across, host/device``
- Claude Code answer excerpt: ``To understand why IPPL uses `Kokkos::View` for field storage, let's explore the relevant code and documentation. This question seems to be about a specific design choice in the IPPL benchmark project. First, we need to identify where `Kokkos::View` is used in the repository. According to the provided context and fil...`

par_003 - How does IPPL parallelize loops over field indices using Kokkos RangePolicy?

- Verdict: `**Partial**`
- Correct reference answer: Loops are parallelized through ``ippl::getRangePolicy`` / ``createRangePolicy`` and the ``ippl::parallel_for`` wrapper over Kokkos ``RangePolicy`` / ``MDRangePolicy``.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.30``
- Reference identifier/location hits: ``1/5``
- Latency: ``5.54s``
- Matched reference signals: ``loops, kokkos, rangepolicy, RangePolicy``
- Missing/weak reference signals: ``parallelized, through, ippl::getrangepolicy, createrangepolicy, ippl::parallel_for, wrapper, mdrangepolicy, MDRangePolicy``
- Claude Code answer excerpt: ``To answer this question about how IPPL uses Kokkos to parallelize loops over field indices, I'll need to explore the source code under `data/raw/ippl`. Let's start by looking at some of the relevant files and symbols. First, I'll use the Read tool to open and inspect the necessary files: ``bash Read`

data/raw/ippl/s...`

par_004 - How does IPPL support mixed precision (float vs double) for fields?

- Verdict: ****Partial****
- Correct reference answer: Precision is chosen by the field value type (``Field<float,...>`` vs ``Field<double,...>``), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.28``
- Reference identifier/location hits: ``0/2``
- Latency: ``3.15s``
- Matched reference signals: ``precision, float, double, codebase, use``
- Missing/weak reference signals: ``chosen, field, value, type, contains, mixed-precision, examples, fields/particles``
- Claude Code answer excerpt: ``To answer this question, I'll need to explore the codebase under `data/raw/ippl` and use the available tools. Let's start by identifying relevant files and symbols that discuss mixed precision in IPPL. ### Step 1: Identify Relevant Files I'll search through the source code for mentions of "mixed precision," "float," "...``

par_005 - How does IPPL handle multiple execution spaces like OpenMP and CUDA in the same build?

- Verdict: ****Partial****
- Correct reference answer: Multiple execution spaces are enabled at build time through ``IPPL_PLATFORMS`` and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.33``
- Reference identifier/location hits: ``0/1``
- Latency: ``5.91s``
- Matched reference signals: ``multiple, execution, spaces, enabled, build``
- Missing/weak reference signals: ``time, through, ippl_platforms, kokkos/heffte, backend, flags, has, container``
- Claude Code answer excerpt: ``To answer how IPPL handles multiple execution spaces like OpenMP and CUDA in the same build, we need to examine the source code of the IPPL library. Let's start by looking at the `data/raw/ippl/include/IPPLConfig.h` file, which contains configuration macros for different execution spaces. 1. **OpenMP Execution Space...``

par_006 - What is the purpose of ParallelDispatch.h and what helpers does it expose?

- Verdict: ****Partial****
- Correct reference answer: ``ParallelDispatch.h`` wraps Kokkos policies and functors, exposing ``RangePolicy``, ``getRangePolicy``, ``createRangePolicy``, ``parallel_for``, and ``parallel_reduce``.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.09``
- Reference identifier/location hits: ``1/6``
- Latency: ``6.49s``
- Matched reference signals: ``paralleldispatch.h, ParallelDispatch.h``
- Missing/weak reference signals: ``wraps, kokkos, policies, functors, exposing, rangepolicy, getrangepolicy, createrangepolicy``
- Claude Code answer excerpt: ``To answer this question, I'll need to explore the source code in `data/raw/ippl`. Specifically, I'll look at the file `ParallelDispatch.h` and any related files that provide context. ### Step 1: Identify `ParallelDispatch.h` First, let's locate `ParallelDispatch.h`: ``bash # Use the FileSearch tool to find Parallel...``

par_007 - How does the Archive class serialize/deserialize Kokkos views for MPI transfer?

- Verdict: ****Partial****
- Correct reference answer: ``Archive`` serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and ``memcpy``, then deserializes them back into views on receive.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.31``
- Reference identifier/location hits: ``1/2``
- Latency: ``6.73s``
- Matched reference signals: ``archive, kokkos, views, Archive``
- Missing/weak reference signals: ``serializes, one-dimensional, byte, buffers, kernels, memcpy, then, deserializes``
- Claude Code answer excerpt: ``To answer this question, we need to explore the codebase and find where the `Archive` class handles serialization and deserialization of Kokkos views for MPI transfer. Let's break down the steps: 1. **Identify Relevant Files**: We'll start by identifying files that might be related to serialization,`

deserialization,...`

par_008 - How are MPI tags managed in IPPL to avoid collisions across subsystems?

- Verdict: ****Partial****
- Correct reference answer: MPI tags are managed with fixed base tags in `Tags.h` and the cyclic tag allocator in `TagMaker`, which `Communicator` inherits.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.38`
- Reference identifier/location hits: `0/3`
- Latency: `5.28s`
- Matched reference signals: `mpi, tags, managed, tag`
- Missing/weak reference signals: `fixed, base, tags.h, cyclic, allocator, tagmaker, communicator, inherits`
- Claude Code answer excerpt: `To answer this question about how MPI tags are managed in IPPL to avoid collisions across subsystems, we need to explore the source code of IPPL located under `data/raw/ippl`. 1. **\*\*Identify relevant files\*\***: We'll start by looking for files that might contain information about MPI tags and their management. 2. **\*\*Rea...**`

par_009 - What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?

- Verdict: ****Partial****
- Correct reference answer: `Kokkos::fence()` waits for outstanding kernels; IPPL exposes `ippl::fence()` as a runtime wrapper in `Ippl.cpp` and also fences internally after many kernel/data-movement steps.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.27`
- Reference identifier/location hits: `4/6`
- Latency: `6.36s`
- Matched reference signals: `kokkos::fence, ippl::fence, wrapper, after, Kokkos::fence, fence(), ippl::fence()`
- Missing/weak reference signals: `waits, outstanding, kernels, exposes, runtime, ippl.cpp, fences, internally`
- Claude Code answer excerpt: `To answer the question "What does Kokkos::fence do, and when does IPPL call the wrapper ippl::fence?", we need to explore the source code of the repository. **### Step 1: Understand Kokkos::fence** Kokkos is a parallel programming library that provides abstractions for expressing computation on heterogeneous systems. `K...`

Boundary And Halo

bc_001 - What boundary condition types are provided in BcTypes.h?

- Verdict: ****Correct****
- Correct reference answer: `BcTypes.h` provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.12`
- Reference identifier/location hits: `1/1`
- Latency: `1.91s`
- Matched reference signals: `bctypes.h, BcTypes.h`
- Missing/weak reference signals: `periodic, constant, zero, extrapolating, no-bc, face, classes`
- Claude Code answer excerpt: `To answer this question, I need to explore the `data/raw/ippl` directory for relevant files and symbols. Let's start by searching for mentions of `BcTypes.h` within that directory. `bash grep -r "BcTypes.h" data/raw/ippl/` This command will recursively search through all files in `data/raw/ippl` for the string...`

bc_002 - How does a periodic boundary condition differ from a Neumann boundary condition in IPPL?

- Verdict: ****Partial****
- Correct reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit `NeumannFace` class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: `0.35`
- Reference identifier/location hits: `0/1`
- Latency: `5.36s`
- Matched reference signals: `periodic, wrap, values, opposite, side, domain`
- Missing/weak reference signals: `bcs, there, explicit, neumannface, codebase, closest, non-periodic, behavior`
- Claude Code answer excerpt: `To answer this question about periodic and Neumann boundary conditions in IPPL (Illinois Plasma Physics Library), we need to explore the relevant files and documentation within the repository. **### Relevant Files** 1. **\*\*File: `src/fields/boundary_conditions.h`\*\*** - This file contains declarations for various boundary co...`

bc_003 - How are ghost cells populated during a halo exchange?

- Verdict: ****Partial****

- Correct reference answer: Ghost cells are populated mainly by ``fillHalo()`` / ``PeriodicFace::apply()``: source slabs are packed, exchanged, and unpacked into ghost regions; ``accumulateHalo_noghost()`` is a different operation.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: ``0.31``

- Reference identifier/location hits: ``0/5``

- Latency: ``8.88s``

- Matched reference signals: ``ghost, cells, populated, exchanged``

- Missing/weak reference signals: ``mainly, fillhalo, periodicface::apply, source, slabs, packed, unpacked, regions``

- Claude Code answer excerpt: ``Ghost cells are essential for accurately simulating physical phenomena in computational fluid dynamics (CFD) and other numerical simulations. During a halo exchange, data is exchanged between the local processor's grid and its neighboring processors' grids to ensure that each processor has sufficient information abo...``

bc_004 - How are physical-cell values copied into ghost cells for periodic boundary conditions?

- Verdict: ****Partial****

- Correct reference answer: Periodic ghost cells are filled from opposite-side physical cells in ``applyPeriodicSerialDim()`` and the multi-rank ``PeriodicFace::apply()`` path.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: ``0.45``

- Reference identifier/location hits: ``0/4``

- Latency: ``10.52s``

- Matched reference signals: ``periodic, ghost, cells, physical``

- Missing/weak reference signals: ``filled, opposite-side, applyperiodicserialdim, multi-rank, periodicface::apply, path, PeriodicFace::apply, PeriodicFace::apply()``

- Claude Code answer excerpt: ``To answer how physical-cell values are copied into ghost cells for periodic boundary conditions in the IPPL codebase, we need to explore the relevant files and symbols. Let's start by identifying the key components involved. ### Step 1: Identify Relevant Files The IPPL codebase is likely structured around several ke...``

bc_005 - How does the absorbing boundary condition in the MaxwellSolvers module damp outgoing waves?

- Verdict: ****Partial****

- Correct reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp outgoing waves.

- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.

- Reference-keyword coverage: ``0.33``

- Latency: ``5.30s``

- Matched reference signals: ``absorbing, boundary, damp, outgoing, waves``

- Missing/weak reference signals: ``maxwell, second-order, mur-style, formulas, precomputed, weights, previous/current/next, values``

- Claude Code answer excerpt: ``To answer this question, we need to explore the source code of the `MaxwellSolvers` module and understand how the absorbing boundary conditions (ABCs) are implemented to damp outgoing waves. Let's break down the steps: 1. **Locate the MaxwellSolvers Module:** - The `MaxwellSolvers` module is likely a part of the sim...``

Numerical Meaning

num_001 - What does the FFT-based Poisson solver physically compute in terms of Coulomb's law?

- Verdict: ****Correct****

- Correct reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to $-\Delta\phi = \rho$.

- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.

- Reference-keyword coverage: ``0.53``

- Reference identifier/location hits: ``0/1``

- Latency: ``10.04s``

- Matched reference signals: ``fft, poisson, compute, electrostatic, potential, charge, coulomb``

- Missing/weak reference signals: ``solvers, optionally, field/gradient, density, i.e, discretized, corresponding, $-\Delta\phi = \rho`$`

- Claude Code answer excerpt: ``To answer the question about what an FFT-based Poisson solver computes in terms of Coulomb's law, we need to explore the source code and documentation for clues. ### Step 1: Understand the Problem Coulomb's law describes the electrostatic interaction between charged particles. The Poisson equation is`

a partial diffe...`

num_002 - What is the physical meaning of the fields advanced by the FDTD solver?

- Verdict: ****Correct****
- Correct reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields ('E', 'B') driven by a source/four-current field.
- Explanation: The answer captures the central facts of the Codex reference and includes the important technical nouns, symbols, or implementation locations.
- Reference-keyword coverage: `0.73`
- Latency: `6.30s`
- Matched reference signals: `maxwell, evolve, electromagnetic, field, electric, magnetic, fields`
- Missing/weak reference signals: `solvers, driven, source/four-current`
- Claude Code answer excerpt: `To answer the question about the physical meaning of the fields advanced by the FDTD solver, we need to explore the source code and documentation related to the Finite-Difference Time-Domain (FDTD) simulation. ### Step-by-Step Analysis: 1. **\*\*Identify Relevant Files:\*\*** - **\*\*File:\*\*** `data/raw/ippl/src/solvers/fdtd_solv...`

num_007 - What are the numerical tradeoffs between the FFT and CG Poisson solvers in IPPL?

- Verdict: ****Incorrect****
- Correct reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but it is more flexible for alternative discretizations, BCs, and preconditioning.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.13`
- Latency: `2.76s`
- Matched reference signals: `fft, poisson, fast`
- Missing/weak reference signals: `direct, structured, uniform, grids, compatible, bcs, but, less`
- Claude Code answer excerpt: `To answer this question, I'll need to explore the source code of the IPPL library to understand the implementation details of both the FFT (Fast Fourier Transform) and CG (Conjugate Gradient) Poisson solvers. Let's start by identifying the relevant files. 1. ****FFT Solver****: The FFT solver is likely implemented in a...`

Build And Install

build_001 - Which third-party libraries does IPPL depend on, and what are they used for?

- Verdict: ****Incorrect****
- Correct reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.11`
- Latency: `8.08s`
- Matched reference signals: `third-party, dependencies`
- Missing/weak reference signals: `core, mpi, distributed, communication, kokkos, performance, portability, heffte`
- Claude Code answer excerpt: `To answer this question, I'll need to explore the source code in `data/raw/ippl` and identify any third-party libraries that it depends on. Let's start by examining the project structure and documentation. ### Step 1: Review Project Structure The IPPL project seems to be structured into several directories and files...`

build_002 - What C++ standard does IPPL require, and why?

- Verdict: ****Incorrect****
- Correct reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Explanation: The answer does not recover the main Codex reference point or points to insufficiently related implementation details.
- Reference-keyword coverage: `0.00`
- Latency: `1.83s`
- Missing/weak reference signals: `requires, c++20, stated, readme, badge, installation, examples/configuration, flags`
- Claude Code answer excerpt: `To determine the C++ standard required by IPPL, I'll need to explore the `data/raw/ippl` directory and its documentation. First, let's check if there are any configuration files or READMEs that mention the C++ standard. We can use the `Search` tool for this: `` <search> query: C++ standard directory: data/raw/ippl...`

build_003 - How is IPPL configured with CMake to enable a GPU build?

- Verdict: ****Partial****
- Correct reference answer: GPU builds are enabled through CMake options like ``IPPL_PLATFORMS=CUDA`` or ``IPPL_PLATFORMS="HIP;OPENMP"`` plus matching Kokkos/architecture/backend flags (``Kokkos_ARCH_*``, ``CMAKE_CUDA_ARCHITECTURES``, ``Kokkos_ENABLE_HIP``, ``Hefhte_ENABLE_ROCM``, etc.).
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.35``
- Reference identifier/location hits: ``0/6``
- Latency: ``4.71s``
- Matched reference signals: ``gpu, builds, enabled, cmake, like, cuda, flags``
- Missing/weak reference signals: ``through, options, ippl_platforms, hip, openmp, matching, kokkos/architecture/backend, kokkos_arch_``
- Claude Code answer excerpt: ``To answer how IPPL (Interdisciplinary Physics and Particle Laboratory) is configured with CMake for a GPU build, we need to look at the relevant CMake configuration files in the `data/raw/ippl` directory. ### Steps to Find the Configuration: 1. **Navigate to CMake Configuration Files**:` - The main CMake configuratio...

build_004 - What options does INSTALLATION.md describe for configuring Kokkos for IPPL?

- Verdict: ****Partial****
- Correct reference answer: ``INSTALLATION.md`` describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and ``IPPL_PLATFORMS`` combinations.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.15``
- Reference identifier/location hits: ``1/2``
- Latency: ``1.52s``
- Matched reference signals: ``installation.md, kokkos, INSTALLATION.md``
- Missing/weak reference signals: ``describes, version, selection, backend/architecture, options, builds, cuda, hip/openmp``
- Claude Code answer excerpt: ``To answer this question, I'll need to explore the `INSTALLATION.md` file and look for any information related to configuring Kokkos for IPPL. Please confirm if you have access to the `INSTALLATION.md` file in the `data/raw/ippl` directory. If so, I can read it and provide you with the relevant information.``

build_005 - How do I install IPPL on the EULER cluster according to the documentation?

- Verdict: ****Partial****
- Correct reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMPI, clone the repo, configure a build, compile, and test on an interactive node.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.27``
- Latency: ``6.09s``
- Matched reference signals: ``euler, install, guide, clone, configure, build``
- Missing/weak reference signals: ``says, connect, vpn/ssh, switch, new, software, stack, purge``
- Claude Code answer excerpt: ``To install IPPL on the EULER cluster according to the provided documentation, you need to follow these steps: 1. **Clone the Repository**:` First, you need to clone the IPPL repository into your working directory. Navigate to ``/psi/home/ext-nobile_a/rag-scientific-code-agent/data/raw/ippl`` and run the following comma...

Testing And Workflow

test_002 - What naming conventions does WORKFLOW.md require for variables, constants, and member v

- Verdict: ****Partial****
- Correct reference answer: ``WORKFLOW.md`` requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with ``_m``.
- Explanation: The answer contains some relevant information but also indicates uncertainty or inability to fully answer.
- Reference-keyword coverage: ``0.50``
- Reference identifier/location hits: ``1/2``
- Latency: ``1.52s``
- Matched reference signals: ``workflow.md, requires, variables, constants, member, WORKFLOW.md``
- Missing/weak reference signals: ``camelcase, capital_case, compile-time, names, suffixed, _m``
- Claude Code answer excerpt: ``WORKFLOW.md` requires specific naming conventions for variables, constants, and member variables. However, it appears that the file is either missing or not checked into the repository. To answer this question accurately, I would need to locate and review the WORKFLOW.md file. Since the file is not available, I cannot...`

test_003 - When should code use Kokkos::sqrt versus std::sqrt according to WORKFLOW.md?

- Verdict: ****Partial****
- Correct reference answer: Use ``std::sqrt`` in host-only code, and ``Kokkos::sqrt`` in device-capable code such as ``KOKKOS_INLINE_FUNCTION`` kernels.
- Explanation: The answer contains some relevant information but also indicates uncertainty or inability to fully answer.
- Reference-keyword coverage: ``0.38``
- Reference identifier/location hits: ``2/3``
- Latency: ``5.47s``
- Matched reference signals: ``use, std::sqrt, kokkos::sqrt, Kokkos::sqrt``
- Missing/weak reference signals: ``host-only, device-capable, such, kokkos_inline_function, kernels, KOKKOS_INLINE_FUNCTION``
- Claude Code answer excerpt: ``To determine when code should use `Kokkos::sqrt` versus `std::sqrt`, we need to examine the documentation and codebase for any guidelines or explanations provided by the IPPL project. **Steps to follow:** 1. **Locate WORKFLOW.md:** This file is likely to contain information about coding conventions and best practice...``

test_004 - How does IPPL profile MPI calls using mpiP?

- Verdict: ****Partial****
- Correct reference answer: IPPL profiles MPI calls with mpiP by preloading ``libmpiP.so``, optionally setting ``MPIP`` flags, then parsing the produced ``mpiP`` output with the bundled ``scripts/mpiP.py``.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.35``
- Reference identifier/location hits: ``2/5``
- Latency: ``4.54s``
- Matched reference signals: ``profiles, mpi, calls, mpip, MPIP, mpiP``
- Missing/weak reference signals: ``preloading, libmpip.so, optionally, setting, flags, then, parsing, produced``
- Claude Code answer excerpt: ``To answer the question "How does IPPL profile MPI calls using mpiP?", we need to explore the source code in the repository. Let's start by looking at relevant files and symbols. ### Step 1: Identify Relevant Files First, let's identify which files might contain information about profiling MPI calls using mpiP. We ca...``

test_005 - What does the SLURM job template for Gwendolen in the README look like?

- Verdict: ****Partial****
- Correct reference answer: The README does include a Gwendolen SLURM template showing the ``gmerlin6`` cluster, ``gwendolen`` partition, GPU count, and ``srun ./<your_executable> <args> --kokkos-map-device-id-by=mpi_rank``.
- Explanation: The answer overlaps with the Codex reference, but misses important scope, implementation detail, or exact terminology.
- Reference-keyword coverage: ``0.44``
- Reference identifier/location hits: ``1/3``
- Latency: ``2.95s``
- Matched reference signals: ``readme, gwendolen, slurm, template, cluster, partition``
- Missing/weak reference signals: ``showing, gmerlin6, gpu, count, srun, your_executable, args, kokkos-map-device-id-by``
- Claude Code answer excerpt: ``The SLURM job template for Gwendolen in the README can be found in `run_eval_ssh.sh`. Here is the relevant section of the file: ``bash #!/bin/bash #SBATCH --partition=gwendolen #SBATCH --gres=gpu:4 #SBATCH --output=slurm-%J.out #SBATCH --error=slurm-%J.err module load gcc/11.3.0 source /Users/aaron/semester_project...``